Week 5: Payment Transaction Management System

Organic Grocery Store Management System

1. Scenario

The Organic Grocery Store needs to maintain payment details for customer orders. The system records payment methods, transaction amounts, and payment statuses. It helps the store identify successful and failed transactions and analyse revenue from different payment methods.

2. Create Payment Table

```
SQL> CREATE TABLE Payment (Payment_ID NUMBER PRIMARY KEY,Order_ID NUMBER(38),Payment_Date DATE,Payment_Mode VARCHAR2(20),Payment_Status VARCHAR2(10),Transac
tion_Amount NUMBER(10,2));

Table created.
```

3.Insert values:

```
SQL>  INSERT INTO Payment VALUES (1, 101, DATE '2026-09-01', 'UPI', 'Success
', 500);

1 row created.

SQL> INSERT INTO Payment VALUES (2, 102, DATE '2026-09-02', 'Card', 'Failed'
, 750)
  2
SQL> INSERT INTO Payment VALUES (2, 102, DATE '2026-09-02', 'Card', 'Failed'
, 750);

1 row created.

SQL> INSERT INTO Payment VALUES (3, 103, DATE '2026-09-03', 'Cash', 'Success
', 300);

1 row created.

SQL> INSERT INTO Payment VALUES (4, 104, DATE '2026-09-04', 'Net Banking', '
Pending', 900);

1 row created.

SQL> INSERT INTO Payment VALUES (5, 105, DATE '2026-09-05', 'UPI', 'Success'
, 400);
```

4.Store payment details:

```
SQL> SELECT * FROM Payment;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_ST
---------- ---------- --------- -------------------- ----------
TRANSACTION_AMOUNT
------------------
         1        101 01-SEP-26 UPI                  Success
               500

         2        102 02-SEP-26 Card                 Failed
               750

         3        103 03-SEP-26 Cash                 Success
               300


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_ST
---------- ---------- --------- -------------------- ----------
TRANSACTION_AMOUNT
------------------
         4        104 04-SEP-26 Net Banking          Pending
               900

         5        105 05-SEP-26 UPI                  Success
               400
```

Report 1: Payment Mode Analysis

Find 1:The number of transactions using UPI.

```
SQL> SELECT COUNT(*) AS Total_UPI_Transactions FROM Payment WHERE Payment_Mode = 'UPI';

TOTAL_UPI_TRANSACTIONS
----------------------
                     2
```

Find 2: The number of card payments.

```
SQL> SELECT COUNT(*) AS Total_Card_Payments FROM Payment WHERE Payment_Mode = 'Card';

TOTAL_CARD_PAYMENTS
-------------------
                  1
```

Find 3: The most preferred payment method.

```
SQL> SELECT Payment_Mode, COUNT(*) AS Total FROM Payment GROUP BY Payment_Mode ORDER BY Total DESC;

PAYMENT_MODE              TOTAL
------------------- ----------
UPI                           2
Cash                          1
Net Banking                   1
Card                          1
```

Report 2: Revenue Analysis

Step 1: The total revenue generated from successful payments.

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Success';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_ST
---------- ---------- --------- -------------------- ----------
TRANSACTION_AMOUNT
------------------
         1        101 01-SEP-26 UPI                  Success
               500

         3        103 03-SEP-26 Cash                 Success
               300

         5        105 05-SEP-26 UPI                  Success
               400
```

Step 2: Find failed transactions

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_ST
---------- ---------- --------- -------------------- ----------
TRANSACTION_AMOUNT
------------------
         2        102 02-SEP-26 Card                 Failed
               750
```

Step 3: Count successful and failed payments

```
SQL> SELECT Payment_Status, COUNT(*) AS Total_Transactions FROM Payment WHER
E Payment_Status IN ('Success', 'Failed') GROUP BY Payment_Status;

PAYMENT_ST TOTAL_TRANSACTIONS
---------- ------------------
Failed                      1
Success                     3
```

Step 4: Analyse payment methods

```
SQL> SELECT Payment_Mode, COUNT(*) AS Total_Transactions FROM Payment GROUP
BY Payment_Mode;

PAYMENT_MODE         TOTAL_TRANSACTIONS
-------------------- ------------------
Net Banking                           1
Card                                  1
Cash                                  1
UPI                                   2
```

Step 5: Calculate total revenue

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue FROM Payment WHERE Paym
ent_Status = 'Success';

TOTAL_REVENUE
-------------
         1200
```

Step 6: Calculate average transaction amount

```
SQL> SELECT AVG(Transaction_Amount) AS Average_Amount FROM Payment WHERE Pay
ment_Status = 'Success';

AVERAGE_AMOUNT
--------------
           400
```

Report 3: Customer Payment History

```
SQL> SELECT c.first_name, o.order_id, p.payment_mode, p.transaction_amount,
p.payment_status FROM Customer c JOIN Orders o ON c.customer_id = o.customer
_id JOIN Payment p ON o.order_id = p.order_id;

FIRST_NAME                                          ORDER_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_ST
-------------------- ------------------ ----------
Arun                                             101
UPI                                 500 Success

Priya                                            102
Card                                750 Failed

Arun                                             103
Cash                                300 Success


FIRST_NAME                                          ORDER_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_ST
-------------------- ------------------ ----------
Kavin                                            104
Net Banking                         900 Pending

Priya                                            105
UPI                                 400 Success
```

Result

The Payment Transaction Management System stores payment records and tracks transaction statuses. The reports help the Organic Grocery Store monitor successful and failed payments, identify popular payment methods, calculate revenue, and review customer payment history.

Important: Your existing Orders table must contain order IDs 101–105 before inserting these payment records. If you have already used these payment IDs, check the existing data before inserting again.